Deakin University

Professional Practice in IT

OnTrack Submission

Task 6.2HD Create your DevOps Pipeline

Submitted By:
Phuc Anh Thu Nguyen
s223212228
2024/09/24 00:50

Tutor:
Sharni Sujatha

Outcome	Weight
Engage with processes, tools, and practices associated with agile project management across all phases of the dev-ops lifecycle, and use software tools to contribute to real-world projects at each stage in an effective manner	♦♦♦♦♦
Evaluate, review, and synthesise real-world scenarios to inform discussion and practice of IT, and relate to professional practice, codes of ethics, and principles of intellectual property and its protection.	♦♦♦♦♦

The task is tough but provides many knowledge into devops

September 24, 2024

Produced by Doubtfire

NUTRISYNC CI/CD PIPELINE PROJECT

1. LINK TO DEMO VIDEO

https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=d0011171-e274-4f76-a2e3-b1f400ea5a48

2. PROJECT DESCRIPTION

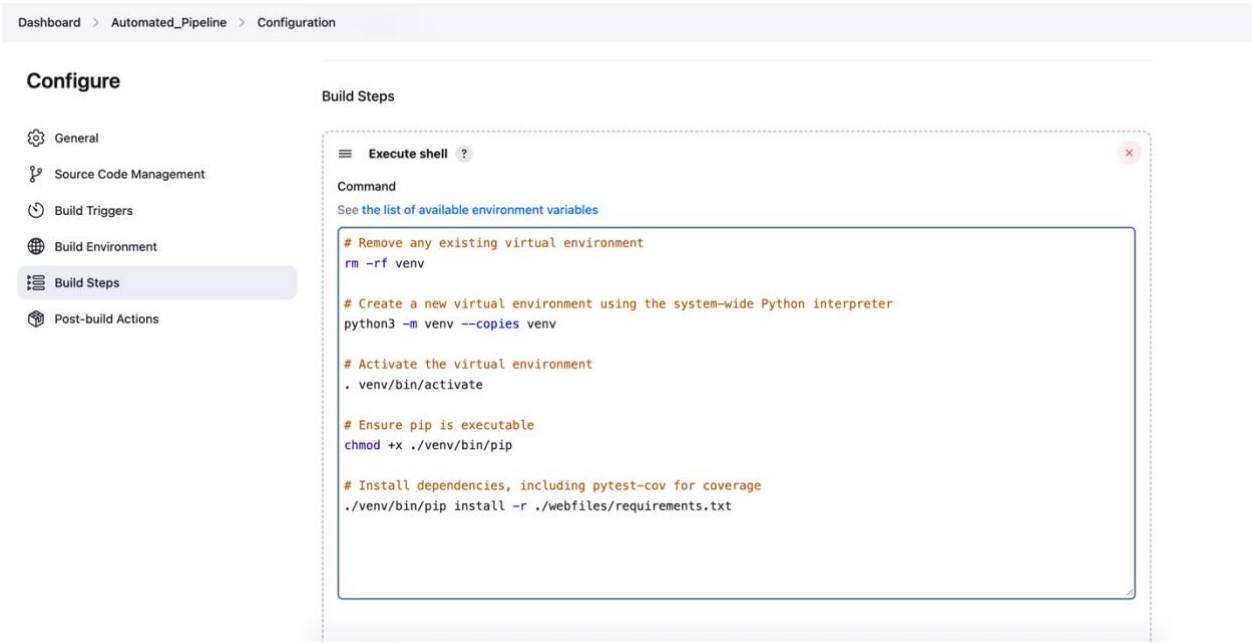
NutriSync is a web application designed to help users sync their health data and manage supplements. The project is built using Python (Flask), with a front-end developed in HTML/CSS. The back-end integrates with databases for storing user information and health metrics.

The goal of this project is to implement a complete **CI/CD pipeline** using **Jenkins** for continuous integration, **Docker** for containerization, and **AWS** for deployment. The pipeline automates the process of building, testing, and deploying the application to both staging and production environments.

TECHNOLOGIES USED:

- **Jenkins**: CI/CD orchestration
- **Docker**: Containerization for building and deploying the application
- **AWS EC2**: Hosting Jenkins, SonarQube, and the application
- **SonarQube**: Code quality analysis
- **GitHub**: Source code management
- **Python (Flask)**: Back-end application framework
- **Pytest**: Testing framework
- **SonarQube**: Code quality and static analysis
- **Docker Hub**: Docker image repository

3. JENKINS PIPELINE SCREENSHOT



See the list of available environment variables

```
# Activate the virtual environment
. venv/bin/activate

# Run pytest with coverage
./venv/bin/pytest --cov=. --cov-report=xml
```

Advanced

Execute SonarQube Scanner

JDK ?

JDK to be used for this SonarQube analysis

(Inherit From Job)

Path to project properties ?

Analysis properties ?

```
sonar.projectKey=SonarQube-App
sonar.sources=./webfiles
sonar.exclusions=**/venv/**,**/tests/**,**/static/**,**/__pycache__/**,**/*.pyc
sonar.python.coverage.reportPaths=coverage.xml
sonar.python.version=3.x
sonar.sourceEncoding=UTF-8
sonar.scm.disabled=true
sonar.python.pylint=/path/to/pylint
sonar.python.pylint.reportPath=pylint-report.txt
```

Execute shell ?

Command

See the list of available environment variables

```
# Navigate to the directory where the docker-compose.yml file is located
cd /var/lib/jenkins/workspace/Automated_Pipeline/webfiles

# Stop and remove existing containers if they are running
docker-compose down || true

# Pull the latest images (if any updates are made)
docker-compose pull

# Deploy the application using docker-compose
docker-compose up -d
```

Advanced

Execute shell ?

Command

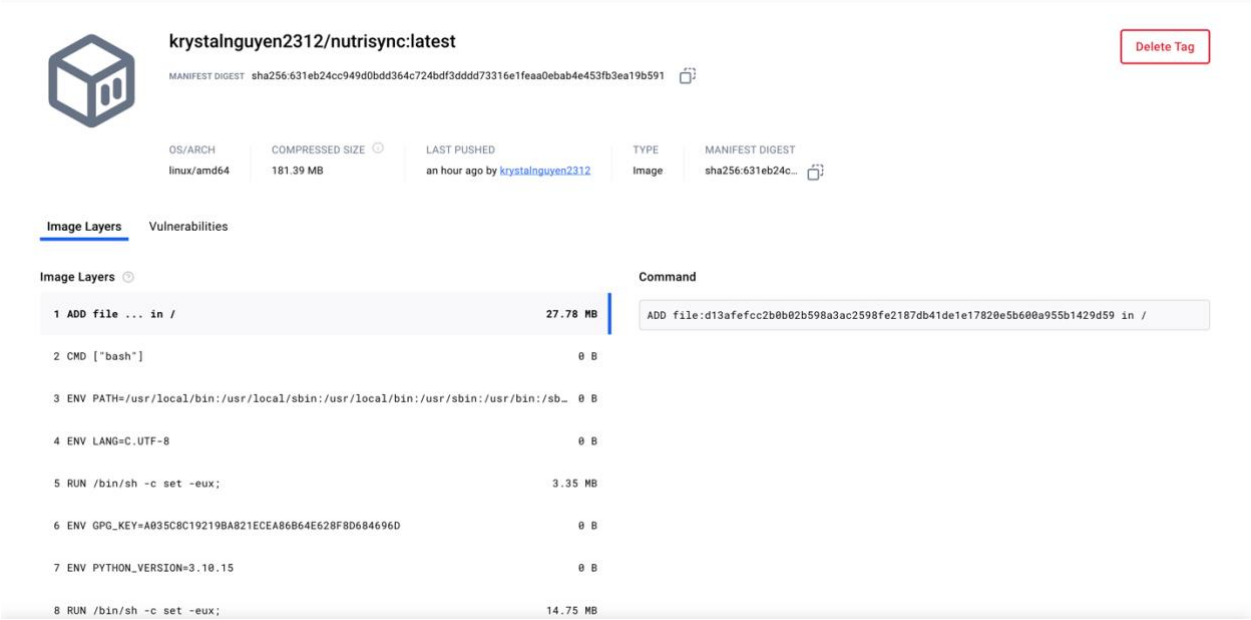
See the list of available environment variables

```
# Navigate to the directory where the docker-compose.prod.yml file is located
cd /var/lib/jenkins/workspace/Automated_Pipeline/webfiles

# Pull the latest changes from GitHub
git pull origin main

# Stop and remove existing production containers (if running)
docker-compose -f docker-compose.prod.yml down || true

# Pull the latest images (if any updates are made)
```

4. DESCRIPTION OF EACH PIPELINE STAGE

BUILD STAGE:

Jenkins creates a Python virtual environment and installs the required dependencies from requirements.txt.
After that, it builds a Docker image using the project's Dockerfile and tags the image as nutrisync:latest.The Docker image is pushed to Docker Hub.



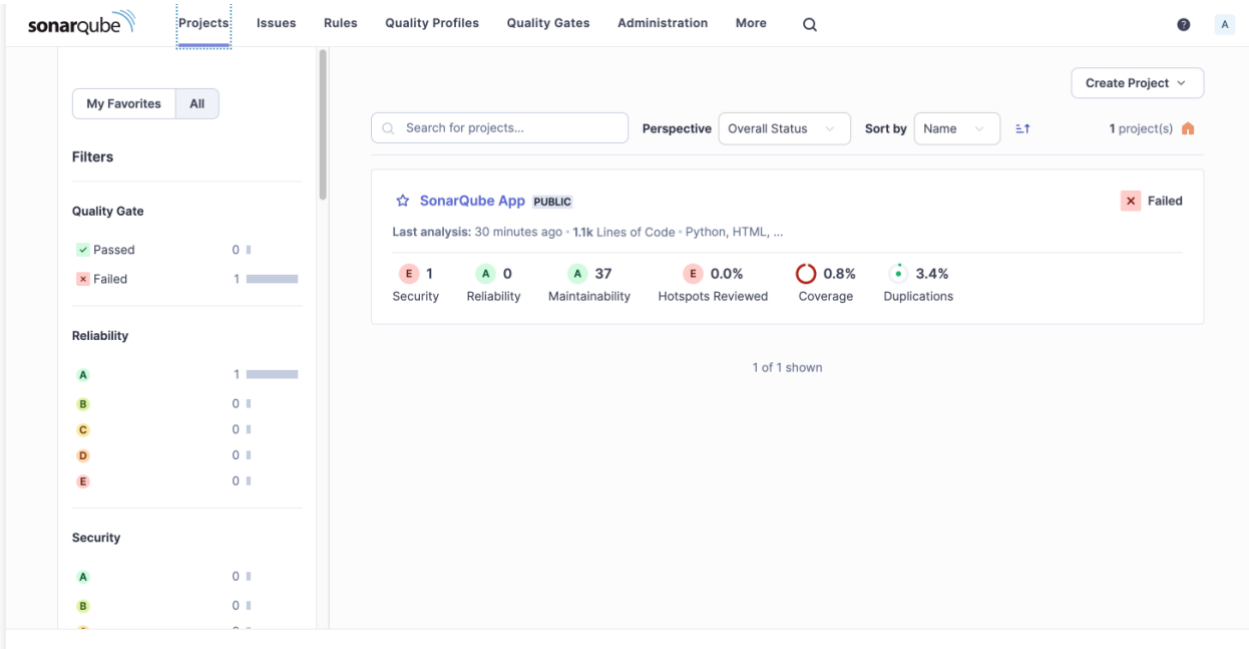
TEST STAGE:

- Pytest is used to run unit tests on the codebase to verify the functionality of different modules. The tests ensure that the core logic of the NutriSync app is working as expected.
- A test coverage report is generated and saved for further analysis.

```
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-8.3.3, pluggy-1.5.0
rootdir: /var/lib/jenkins/workspace/Automated_Pipeline
plugins: cov-5.0.0
collected 2 items

webfiles/test_main.py ..                                                 [100%]

---------- coverage: platform linux, python 3.12.3-final-0 -----------
Coverage XML written to file coverage.xml


============================== 2 passed in 0.05s ===============================
```

CODE QUALITY ANALYSIS STAGE

This stage is integrated with SonarQube. The code is analyzed for code smells, bugs, and security vulnerabilities. A detailed report is generated with metrics such as maintainability, reliability, and security vulnerabilities.



DEPLOY STAGE:

- In this stage, Jenkins uses Docker Compose to deploy the application in a staging environment on AWS EC2. The application container is spun up in this environment for testing purposes.

RELEASE STAGE:

- After successful testing in staging, the application is promoted to the production environment using a separate docker-compose.prod.yml file. Jenkins orchestrates the deployment in the production environment on AWS EC2.

---

5. TESTING FRAMEWORK AND TESTS DESCRIPTION

Pytest was used as the testing framework to ensure the code's functionality before deployment. The tests focused on:

- **Unit tests** for core application logic.
- **Integration tests** for ensuring modules interact correctly.
- A **coverage report** was generated to track how much of the code was covered by these tests.

In this project, the key test files were test_main.py, which covered the critical components of the NutriSync application. As seen from the Jenkins pipeline, all the tests passed successfully.

---

6. DEPLOYMENT TOOL AND TEST ENVIRONMENT

Deployment Tool:

- **Docker** was used as the containerization tool, and **Docker Compose** handled orchestrating the services for the NutriSync application in the test environment.
- Docker Compose pulls the latest image from Docker Hub and spins up the containers in a staging environment on an **AWS EC2 instance**.

Test Environment:

- The test environment was hosted on an **AWS EC2 instance**, running the Docker container where NutriSync was deployed and tested. The test environment simulates the final production environment to ensure no issues arise during the release.

---

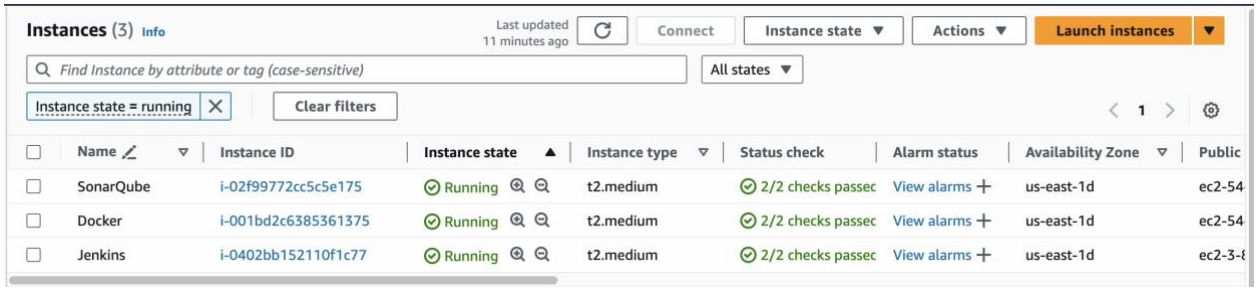
7. RELEASE MANAGEMENT TOOL AND PRODUCTION ENVIRONMENT

Release Management Tool:

- The **docker-compose.prod.yml** file is used for deploying NutriSync to the production environment. Jenkins automatically manages this by pulling the image and starting the application in production mode.

Production Environment:

- The production environment is hosted on another **AWS EC2 instance**, separate from the staging environment. This ensures that the application is isolated and runs with optimal settings for real-world use.



CONCLUSION

This project demonstrates the full automation of the CI/CD pipeline for NutriSync using Jenkins, Docker, and AWS. The pipeline ensures that every code change goes through a rigorous build, test, analysis, and deployment process, improving code quality and reducing time to release new features.